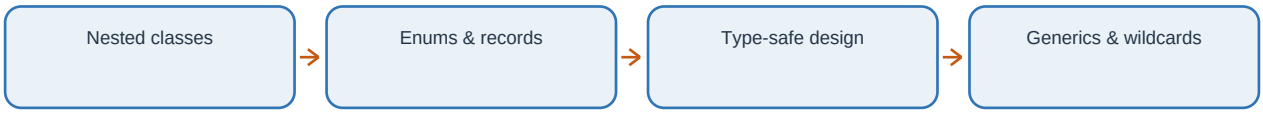
JAVA NOTES

Inner Classes • Enums • Records • Sealed Types • Generics

Complete revision notebook
Definitions, syntax, diagrams, code examples, outputs, practical uses and exam tips.

HOW THESE TOPICS FIT TOGETHER



Java version note: Records became permanent in Java 16 and sealed classes/interfaces in Java 17. Examples use modern Java syntax; generic fundamentals work across many Java versions.

1. Member Inner and Static Nested Classes

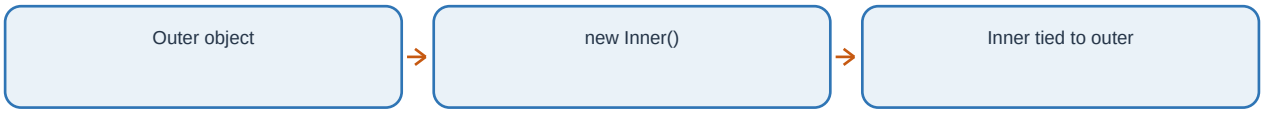
Nested class: a class declared inside another class. It helps group related types, control visibility and keep implementation details close to the code that uses them.

Two important member nested-class forms

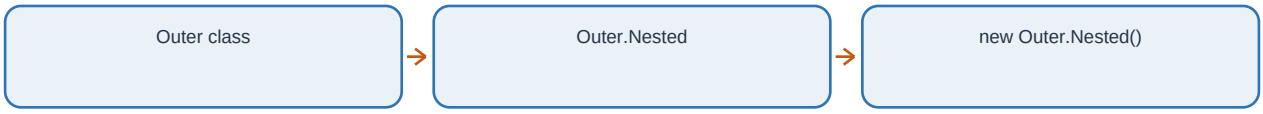
• Member inner class: a non-static class declared directly inside another class. Each instance is associated with an enclosing outer-class instance.

• Static nested class: declared with `static`. It does not need an outer object and cannot directly access the outer object's instance members.

CREATING THE TWO TYPES



STATIC NESTED CLASS




```
class House {
    private String address = "12 Park Road";
    private static int count = 0;

    class Room {                        // member inner class
        void show() {
            System.out.println(address); // accesses outer instance field
        }
    }

    static class Blueprint {           // static nested class
        void showCount() {
            System.out.println(count);  // accesses static field
        }
    }
}

class Demo {
    public static void main(String[] args) {
        House h = new House();
        House.Room room = h.new Room();
        room.show();

        House.Blueprint plan = new House.Blueprint();
        plan.showCount();
    }
}
```

Output

```
12 Park Road\n0
```

Key rules

• Create an inner object using `outerObject.new Inner()`.

• Create a static nested object using `new Outer.Nested()`.

• An inner class can access private members of its enclosing object. A static nested class can directly access only static members.

• A static nested class may still access an outer instance member when it is given an outer object reference.

Real-world use: a Map.Entry-style helper type, a builder, or a class that represents a component owned by one outer object.

2. Method-local and Anonymous Classes

Local class: a class declared inside a method, constructor or block. Its name is visible only within that declaration scope.

Anonymous class: an unnamed class expression used to create one object and override or implement behavior immediately.

```
class Printer {
    void printLater() {
        final String message = "Hello from local class";

        class Job {                 // local class
            void run() {
                System.out.println(message);
            }
        }
        new Job().run();

        Runnable task = new Runnable() { // anonymous class
            @Override
            public void run() {
                System.out.println("Task executed");
            }
        };
        task.run();
    }
}
```

Output

```
Hello from local class\nTask executed
```

Effectively final: a local variable used by a local or anonymous class must be final or effectively final. Effectively final means it is assigned once and never reassigned.

```
int limit = 5;                  // effectively final
Runnable r = new Runnable() {
    public void run() {
        System.out.println(limit);
    }
};
// limit = 6;                   // would make the capture illegal
```

When to use

• Local class: a helper implementation needed only in one method.

• Anonymous class: a one-off implementation of an interface or subclass, especially in older callback-style APIs.

• For a single-method interface, a lambda is usually shorter; a lambda is not itself an anonymous class.

Remember: anonymous classes can declare additional fields and methods; lambdas cannot introduce a new class body with extra members.

3. Outer Members, Callbacks and this

A nested class can use the enclosing object's members. When names collide, qualify the outer member with `OuterClass.this`.

```
class Screen {
    private String name = "Main screen";

    class Button {
        private String name = "Save";

        void click() {
            System.out.println(name);             // Button.name
            System.out.println(Screen.this.name); // outer Screen.name
        }
    }
}
```

Output

```
Save\nMain screen
```

Callback: passing an object or function-like behavior so another part of the program can notify you when an event occurs. Inner classes and lambdas are common callback implementations.

```
interface ClickListener {
    void onClick();
}

class Window {
    void setListener(ClickListener listener) {
        // Store it and invoke it later when the button is clicked.
        listener.onClick();
    }
}

class App {
    void start() {
        Window w = new Window();
        w.setListener(() -> System.out.println("Button clicked"));
    }
}
```

this refers to the current object. In an inner class, `this` is the inner object; `Outer.this` is the enclosing object.

Practical caution: callbacks that retain an inner-class instance may also retain its outer object. Avoid unintentionally keeping large or short-lived outer objects alive.

4. Enum Constants, Fields and Constructors

Enum: a special Java type representing a fixed set of named constants. Use it when a value must come from a known, limited set.

```
enum OrderStatus {
    NEW("Order received"),
    PACKED("Ready to ship"),
    SHIPPED("On the way"),
    DELIVERED("Completed");

    private final String description;

    OrderStatus(String description) { // enum constructor
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}

class Demo {
    public static void main(String[] args) {
        OrderStatus status = OrderStatus.SHIPPED;
        System.out.println(status);
        System.out.println(status.getDescription());
    }
}
```

Output

```
SHIPPED\nOn the way
```

Rules

• Enum constants are declared first, separated by commas; end the list with a semicolon if fields, constructors or methods follow.

• An enum constructor cannot be public or protected; it is effectively private.

• Enum instances are fixed singleton constants; do not create them with `new`.

• Enums can implement interfaces and can contain fields, methods and constant-specific class bodies.

Why fields? Store metadata with each constant instead of scattering switch statements or unrelated numeric codes throughout the application.

5. values(), valueOf(), EnumSet and EnumMap

`values()` returns an array containing enum constants in declaration order. `valueOf(String)` returns the constant whose name exactly matches the supplied string.

```
enum Priority { LOW, MEDIUM, HIGH }

for (Priority p : Priority.values()) {
    System.out.println(p);
}

Priority p = Priority.valueOf("HIGH");
System.out.println(p.name());
System.out.println(p.ordinal()); // declaration position: HIGH is 2
```

Output

```
LOW\nMEDIUM\nHIGH\nHIGH\n2
```

Important: `valueOf` is case-sensitive and throws `IllegalArgumentException` for an unknown name. `ordinal()` is a zero-based declaration position; do not use it as a persistent database ID.

EnumSet and EnumMap

- EnumSet: a specialized, efficient Set for values from one enum type.
- EnumMap: a specialized Map whose keys are values from one enum type.

```
import java.util.EnumSet;
import java.util.EnumMap;

enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

class Demo {
    public static void main(String[] args) {
        EnumSet<Day> workDays =
            EnumSet.range(Day.MON, Day.FRI);
        System.out.println(workDays);

        EnumMap<Day, String> plan = new EnumMap<>(Day.class);
        plan.put(Day.MON, "Study Java");
        plan.put(Day.SAT, "Build project");
        System.out.println(plan.get(Day.MON));
    }
}
```

Typical output

```
[MON, TUE, WED, THU, FRI]\nStudy Java
```

Use them when: representing selected permissions, enabled weekdays, workflow states or enum-keyed configuration.

6. Records and Sealed Types Overview

Record: a concise way to declare a class whose main purpose is to carry data. The compiler provides a canonical constructor, accessors, `equals`, `hashCode` and `toString`.

```
record Student(int id, String name) { }

class Demo {
    public static void main(String[] args) {
        Student s = new Student(101, "Asha");
        System.out.println(s.name()); // accessor, not getName()
        System.out.println(s);
    }
}
```

Output

```
Asha\nStudent[id=101, name=Asha]
```

• Record components are final; records are shallowly immutable, so a mutable object stored in a component can still be changed.

• You may add validation in a compact constructor: `Student { if (id <= 0) throw new IllegalArgumentException(); }`.

• A record cannot extend another class; it implicitly extends `java.lang.Record`, but it may implement interfaces.

Sealed class/interface: restricts which classes may directly extend or implement it. The permitted types must declare how they continue the hierarchy: `final`, `sealed` or `non-sealed`.

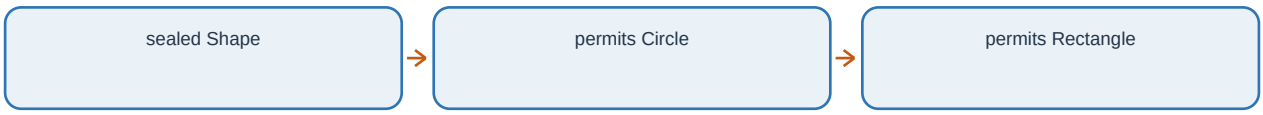
```
sealed interface Shape permits Circle, Rectangle { }

record Circle(double radius) implements Shape { }

final class Rectangle implements Shape {
    final double width, height;
    Rectangle(double w, double h) {
        width = w;
        height = h;
    }
}
```

Diagram

A CONTROLLED TYPE HIERARCHY



Use: records for data carriers such as coordinates or API responses; sealed types for controlled domain models such as a known set of payment or expression types.

7. Generic Classes, Methods and Interfaces

Generics let types be parameters. They improve compile-time type safety and reduce explicit casting.

Generic class

```
class Box<T> {
    private T value;

    void set(T value) { this.value = value; }
    T get() { return value; }
}

Box<String> box = new Box<>();
box.set("Java");
String text = box.get(); // no cast required
```

Here T is a type parameter. Common conventional names include T (type), E (element), K (key), V (value) and N (number).

Generic method

```
class Tools {
    static <T> void printPair(T first, T second) {
        System.out.println(first + " / " + second);
    }
}

Tools.printPair("ID", 42);
```

The type parameter appears before the return type. The compiler can often infer it from the arguments.

Generic interface

```
interface Repository<T> {
    void save(T item);
    T findById(int id);
}

class StudentRepository implements Repository<Student> {
    public void save(Student item) { /* save item */ }
    public Student findById(int id) { return null; }
}
```

Benefits: safer collections, reusable algorithms, clearer APIs and earlier detection of type mismatches.

8. Bounds and Wildcards

Bounded type parameter: restricts the types that may be supplied to a generic declaration.

```
class Calculator<T extends Number> {
    double twice(T value) {
        return value.doubleValue() * 2;
    }
}

Calculator<Integer> c = new Calculator<>();
System.out.println(c.twice(7)); // 14.0
```

`extends` in a type bound means a subtype of the bound; it works for classes and interfaces. Multiple bounds use `&`, with a class bound first.

```
class Item<T extends Number & Comparable<T>> { }
```

Wildcards

• `List<?>`: unknown element type; safe to read as Object, but cannot add a specific non-null value.

• `List<? extends Number>`: an unknown subtype of Number; useful as a producer of Number values. You generally cannot add values (except null).

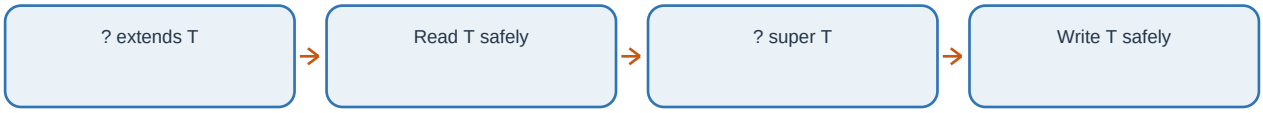
• `List<? super Integer>`: an unknown supertype of Integer; you can safely add Integer values, but reads are only guaranteed as Object.

```
static double total(List<? extends Number> nums) {
    double sum = 0;
    for (Number n : nums) sum += n.doubleValue();
    return sum;
}

static void addDefaults(List<? super Integer> out) {
    out.add(10);
    out.add(20);
}
```

Diagram: direction of safe use

WILDCARD QUICK GUIDE



Do not confuse: `List<Integer>` *is not a subtype of* `List<Number>`*. Generic types are invariant.*

9. Type Erasure and Restrictions

Java implements generics mainly through type erasure: the compiler checks generic types, then removes most type-parameter information from ordinary runtime representations. Unbounded type parameters erase to Object; bounded parameters erase to their first bound.

Example concept

```
class Box<T> {
    T value;
    T get() { return value; }
}

// Conceptually, after erasure for an unbounded T:
// class Box { Object value; Object get() { return value; } }
```

Common restrictions

• You cannot create an instance of a type parameter: `new T()` is illegal.

• You cannot create a generic array directly: `new T[10]` is illegal.

• You cannot use primitive type arguments: use `Integer`, not `int`.

• You cannot overload methods whose signatures become identical after erasure.

• You cannot use `instanceof List<String>`; the runtime cannot check that parameterized type. `instanceof List<?>` is allowed.

• A generic class cannot extend Throwable, so you cannot create generic exception classes.

Why do these restrictions exist?

They preserve compatibility with older Java bytecode and avoid pretending that erased type arguments are available at runtime. Some generic information may remain in class-file metadata for reflection, but it is not generally available for runtime type checks.

Safe design tip: accept a `Class<T>` token or a factory/supplier when code needs to construct T; use carefully checked varargs or collections instead of unsafe generic arrays.

10. PECS Principle

PECS = Producer Extends, Consumer Super. It is a practical rule for choosing wildcard bounds when designing methods that work with generic collections.

Producer — use extends

If a parameter only supplies values to your method, use `? extends T`. The method can read each value as T.

```
static void copyNumbers(List<? extends Number> source,
                        List<? super Number> destination) {
    for (Number n : source) {
        destination.add(n);
    }
}
```

Consumer — use super

If a parameter receives values of type T, use `? super T`. The destination may be a List of T or any supertype of T.

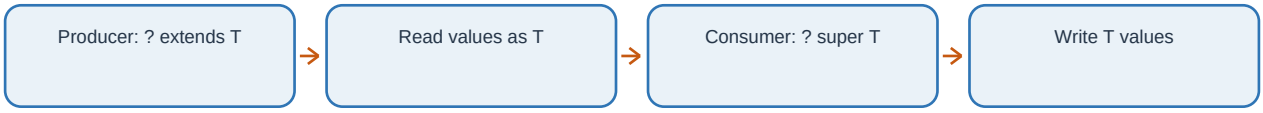
Example

```
List<Integer> source = List.of(1, 2, 3);
List<Number> destination = new ArrayList<>();

copyNumbers(source, destination);
System.out.println(destination); // [1, 2, 3]
```

The source is a producer of Number values because Integer extends Number. The destination is a consumer because it can accept Number values.

PECS AT A GLANCE



Quick decision table

Requirement	Choose	Reason
Read T values	`? extends T`	Producer supplies T or subtype
Write T values	`? super T`	Consumer accepts T or supertype
Read and write same exact type	`T`	Use a type parameter

Memory trick: data comes out of a producer (extends); data goes into a consumer (super).

11. Final Revision Sheet & Practice

One-minute revision

• Member inner class → needs an enclosing object; static nested class → does not.

• Local class → named, method/block scoped; anonymous class → unnamed one-off implementation.

• `Outer.this` identifies the enclosing object; callbacks communicate events or completion.

• Enum → fixed constants with optional fields, constructors and methods.

• `values()` lists constants; `valueOf()` parses an exact constant name.

• EnumSet / EnumMap → specialized set/map for enum values.

• Record → concise data carrier; sealed type → controlled inheritance.

• Generics → compile-time type safety; bounds constrain type arguments; wildcards express flexibility.

• Type erasure → generic parameters are mostly removed from runtime types.

• PECS → Producer Extends, Consumer Super.

Practice questions

1. How do you instantiate a member inner class?
2. Can a static nested class access an outer instance field directly?
3. What is the difference between an anonymous class and a lambda?
4. Why should ordinal() not be stored as a database ID?
5. What methods does a record provide automatically?
6. What does sealed restrict?
7. What is the purpose of a generic method type parameter?
8. Why is List<Integer> not assignable to List<Number>?
9. What does type erasure mean?
10. In PECS, which wildcard is used for a destination that consumes values?

Answer key

1. `outer.new Inner()`. 2. No, it needs an outer object reference. 3. Anonymous class creates a class body; lambda expresses a functional interface implementation. 4. Declaration order can change. 5. Accessors, canonical constructor, equals, hashCode and toString. 6. Permitted direct subtypes. 7. Reusable type-safe method behavior. 8. Generic types are invariant. 9. Type parameters are mostly removed by compilation. 10. `? super T`.

Study tip: compile each code example, change one line at a time, and predict the result before running it. That is the fastest way to master nested classes and generic wildcards.